Premier League Player Statistics (2023–2024 Season)

AI221

نواف مساعد الحربي 4403059
مشاري إبراهيم الرحيلي 4401118
عبدالعزيز فهد المطيري 4402898

The dataset:

Dataset Dimensions:

The dataset contains **581 rows** and **35 columns**.

Key Features:

The dataset likely includes attributes related to Premier League players for the 2023–2024 season. Common features in such datasets might include:

- **Player Name:** Name of the player.
- **Position:** The role (e.g., Forward, Midfielder, Defender, Goalkeeper).
- **Team:** The club the player represents.
- **Appearances:** Number of matches played.
- **Goals:** Number of goals scored.
- **Assists:** Number of assists provided.
- **Minutes Played:** Total minutes played.
- **Yellow Cards/Red Cards:** Disciplinary records.

Potential Uses of the Dataset:

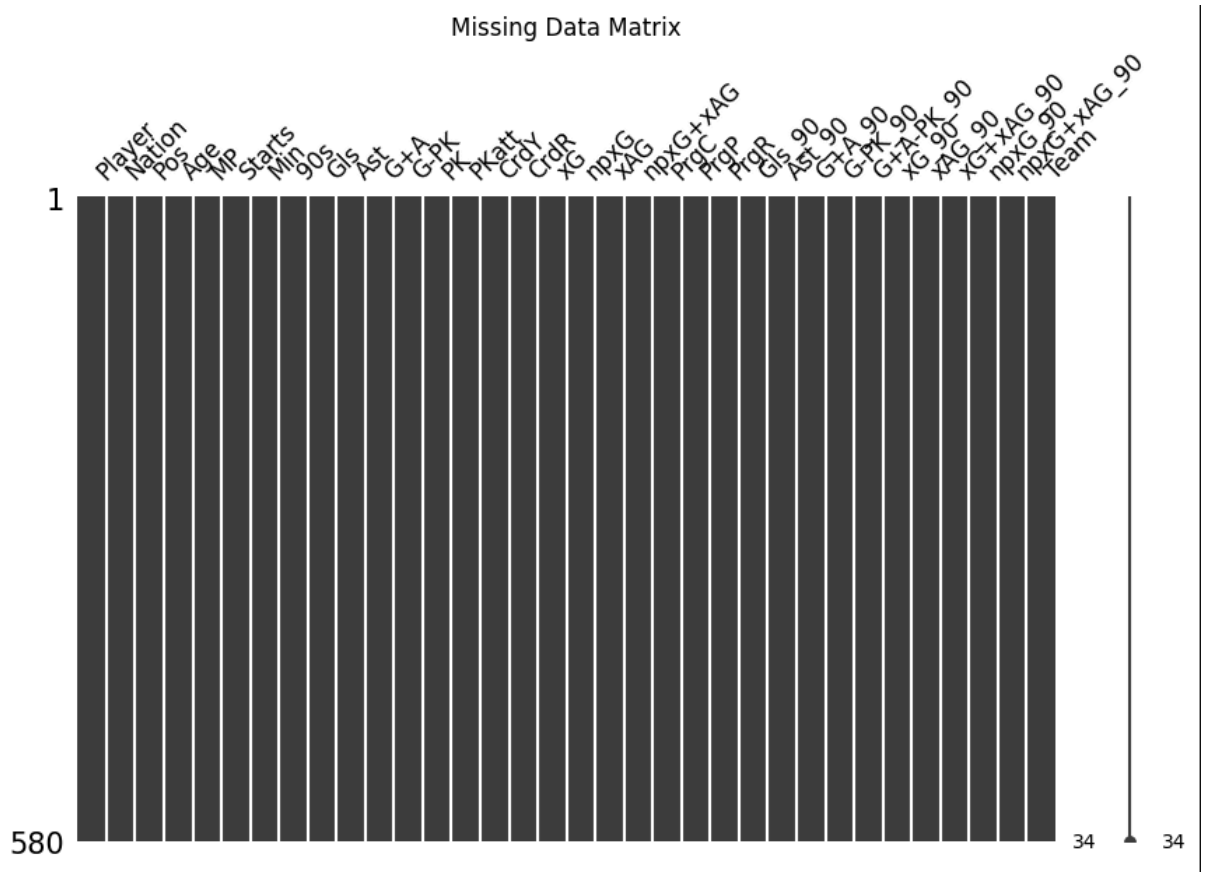
- **Player Performance Analysis**
  - Evaluate individual player contributions in terms of goals, assists, and defensive statistics.
  - Identify top-performing players and their strengths across different teams.
- **Team Strength Comparison**
  - Compare teams based on metrics like average goals, assists, defensive strength, and disciplinary stats.
  - Assess team rankings and performances throughout the 2023–2024 season.
- **Tactical Insights**
  - Analyze player positions and formations to understand team strategies and tactics.

Sample Structure:

	Player	Nation	Pos	Age	MP	Starts	Min	90s	Gls	Ast
0	Rodri	es ESP	MF	27.0	34	34	2931.0	32.6	8.0	9.0
1	Phil Foden	eng ENG	FW,MF	23.0	35	33	2857.0	31.7	19.0	8.0
2	Ederson	br BRA	GK	29.0	33	33	2785.0	30.9	0.0	0.0
3	Julián Álvarez	ar ARG	MF,FW	23.0	36	31	2647.0	29.4	11.0	8.0
4	Kyle Walker	eng ENG	DF	33.0	32	30	2767.0	30.7	0.0	4.0
5	Bernardo Silva	pt POR	MF,FW	28.0	33	29	2578.0	28.6	6.0	9.0
6	Erling Haaland	no NOR	FW	23.0	31	29	2552.0	28.4	27.0	5.0

Data Processing:

1. Checking for missing values:
   - we going to check the dataset for missing values using The missingno library to visualize missing values in the dataset.



this the output after we used missingno library and as we can there is no missing values.

2. Removing duplicate records:

   we will use drop_duplicates() to removes duplicate rows from the dataframe.

3. Converting data types:

   after we used dtypes attribute in pandas shows the data type of each column in a DataFrame.

```
[6]: Player          object
     Nation          object
     Pos             object
     Age            float64
     MP               int64
     Starts           int64
     Min            float64
     90s            float64
     Gls            float64
     Ast            float64
     G+A            float64
     G-PK           float64
     PK             float64
     PKatt          float64
     CrdY           float64
     CrdR           float64
     xG             float64
     npxG           float64
     xAG            float64
     npxG+xAG       float64
     PrgC           float64
     PrgP           float64
     PrgR           float64
```

As we can see, some data types are incorrect, and this has to be fixed. The data we need to change is: Age, Min, Gls, Ast, G+A, G-PK, PK, PKatt, CrdY, CrdR, PrgC, PrgP, and PrgR.

```
[30]: Player          object
      Nation          object
      Pos             object
      Age              int32
      MP               int64
      Starts           int64
      Min              int32
      90s            float64
      Gls              int32
      Ast              int32
      G+A              int32
      G-PK             int32
      PK               int32
      PKatt            int32
      CrdY             int32
      CrdR             int32
      xG             float64
      npxG           float64
      xAG            float64
      npxG+xAG       float64
      PrgC             int32
      PrgP             int32
      PrgR             int32
```

now we move on after we fixed the data types

4. Validate data:

   We will check for negative values four columns and it will be (Age,Goals,xG_90, and minutes) by using this line

```
negative_values = {col: pl[pl[col] < 0].shape[0] for col in ["Age", "Gls", "xG_90", "Min"] if (pl[col] < 0).any()}
```

5. Invalid ranges:

   We are going to check for an invalid age range. We know the biggest player in the Premier League is Thiago Silva at the age of 39, and the youngest player is Mikey Moore at the age of 15.

   we will use this line:

```
invalid_age = pl[(pl['Age'] < 15) | (pl['Age'] > 39)]
```

Data Analysis:

1. Calculate total goals scored by top 5 teams:

we going to Calculate total goals scored by each team by using this line:

```
team_goals = pl.groupby("Team")["Gls"].sum().sort_values(ascending=False)
```

The output:

```
Top 5 teams by total goals scored:
Team
Manchester City     94
Arsenal             86
Newcastle United    83
Liverpool           80
Chelsea             76
```

2. Top 10 players goals:

we want to show a table of the top 10 players with goals but we want also to show with it the expected goals but the 10 players they should have at least 10 goals and to do this we will use this line:

```
top_xg_players = pl[pl['Gls'] >= 10].sort_values('Gls', ascending=False).head(10)
```

```
print(top_xg_players[['Player', 'Team', 'Gls', 'xG']])
```

The output:

```
Top 10 players with the highest xG (minimum 10 goals):
              Player               Team  Gls    xG
6     Erling Haaland    Manchester City   27  29.2
83       Cole Palmer            Chelsea   22  18.2
117   Alexander Isak   Newcastle United   21  20.3
204    Ollie Watkins        Aston Villa   19  16.8
1         Phil Foden    Manchester City   19  10.3
369  Dominic Solanke        Bournemouth   19  19.6
28     Mohamed Salah          Liverpool   18  21.2
146    Son Heung-min  Tottenham Hotspur   17  12.0
237     Jarrod Bowen    West Ham United   16  11.6
59       Bukayo Saka            Arsenal   16  15.5
```

Why do we include the expected goals?

We take into account the expected goals to determine how effective the player is at scoring goals. For example, Haaland and Mohemed Salah require more chances to score goals, but Cole Palmer, Phil Foden, and Son require fewer chances, demonstrating their effectiveness in scoring goals compared to the others.

3. Top 10 players assists:

The same thing but here we will replace the goals with assists and this the line we need:

```
top_xag_players = pl[pl['Ast'] >= 10].sort_values('Ast', ascending=False).head(10)
```

```
print(top_xag_players[['Player', 'Team', 'Ast', 'xAG']])
```

The output:

```
Top 10 players with the highest xAG (minimum 10 assists):
               Player                Team  Ast   xAG
204    Ollie Watkins         Aston Villa   13   7.3
83       Cole Palmer             Chelsea   11  11.1
13   Kevin De Bruyne     Manchester City   10  11.5
28     Mohamed Salah           Liverpool   10  11.8
57   Martin Ødegaard             Arsenal   10   9.6
114   Anthony Gordon    Newcastle United   10   8.0
118  Kieran Trippier    Newcastle United   10   7.5
146    Son Heung-min   Tottenham Hotspur   10  11.8
154  Brennan Johnson   Tottenham Hotspur   10   8.1
338      Pascal Groß            Brighton   10   8.1
```

4. High-Impact Players:

Analyze players with the highest combined contributions (goals + assists) and their impact on their team's success.

```
top_ga_players = pl.sort_values('G+A', ascending=False)[['Player', 'Team', 'G+A', 'Gls', 'Ast']]
```

The output:

	Player	Team	G+A	Gls	Ast
83	Cole Palmer	Chelsea	33	22	11
6	Erling Haaland	Manchester City	32	27	5
204	Ollie Watkins	Aston Villa	32	19	13
28	Mohamed Salah	Liverpool	28	18	10
146	Son Heung-min	Tottenham Hotspur	27	17	10
1	Phil Foden	Manchester City	27	19	8
59	Bukayo Saka	Arsenal	25	16	9
117	Alexander Isak	Newcastle United	23	21	2
369	Dominic Solanke	Bournemouth	22	19	3
237	Jarrod Bowen	West Ham United	22	16	6

5. Underperformers vs Overperformers:

To analyze Underperformers vs. Overperformers, we can compare actual performance metrics (goals and assists) with expected metrics (xG and xAG).

The code**:**

```
pl['xG_diff'] = pl['Gls'] - pl['xG']
pl['xAG_diff'] = pl['Ast'] - pl['xAG']

overperformers = pl[pl['xG_diff'] > 0].sort_values(by='xG_diff', ascending=False).head(5)

underperformers = pl[pl['xG_diff'] < 0].sort_values(by='xG_diff').head(5)
```

The output:

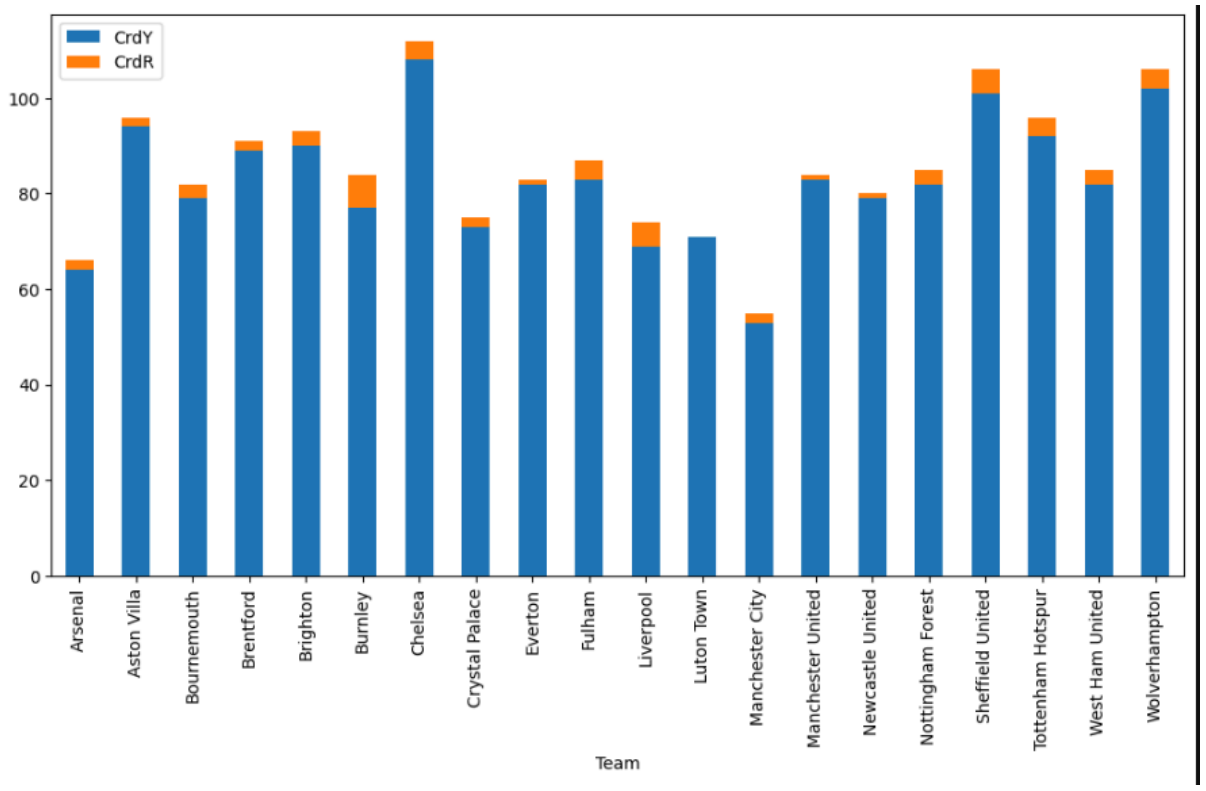
```
Top Overperformers:
                    Player                Team    xG  Gls  xG_diff
1                Phil Foden     Manchester City  10.3   19      8.7
457     Callum Hudson-Odoi  Nottingham Forest   2.7    8      5.3
265  Jean-Philippe Mateta      Crystal Palace  10.9   16      5.1
146          Son Heung-min  Tottenham Hotspur  12.0   17      5.0
39              Diogo Jota           Liverpool   5.3   10      4.7

Top Underperformers:
                     Player                Team    xG  Gls  xG_diff
320  Dominic Calvert-Lewin             Everton  12.9    7     -5.9
32            Darwin Núñez           Liverpool  16.3   11     -5.3
154        Brennan Johnson  Tottenham Hotspur  10.3    5     -5.3
82         Nicolas Jackson             Chelsea  18.6   14     -4.6
26               Luis Díaz           Liverpool  11.9    8     -3.9
```

Data Visualisation:

1. Discipline Analysis:

Discipline analysis examines player and team behavior in terms of yellow cards, and red cards. This can provide insights into the aggressiveness or discipline of teams and players.

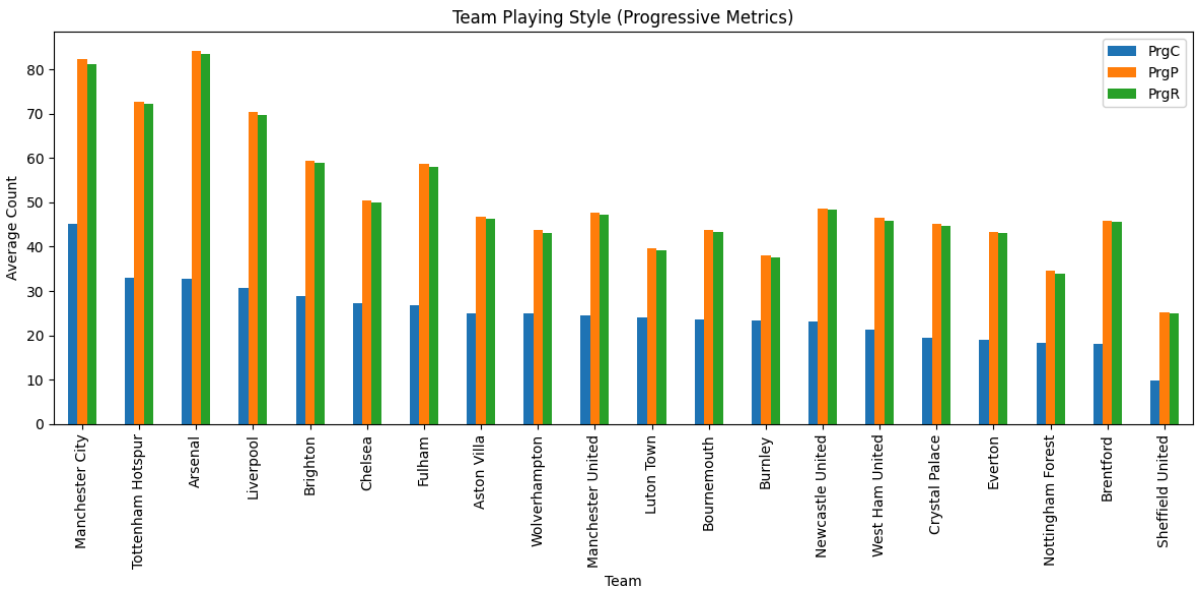


As we can see from the chart the most aggressive team in the league is Chelsea and the most discipline team is Manchester City

2. Team Style of Play (Progressive Metrics):

Progressive metrics like Progressive Carries (PrgC), Progressive Passes (PrgP), and Progressive Receives (PrgR) offer insights into how teams move the ball up the field, indicating their style of play

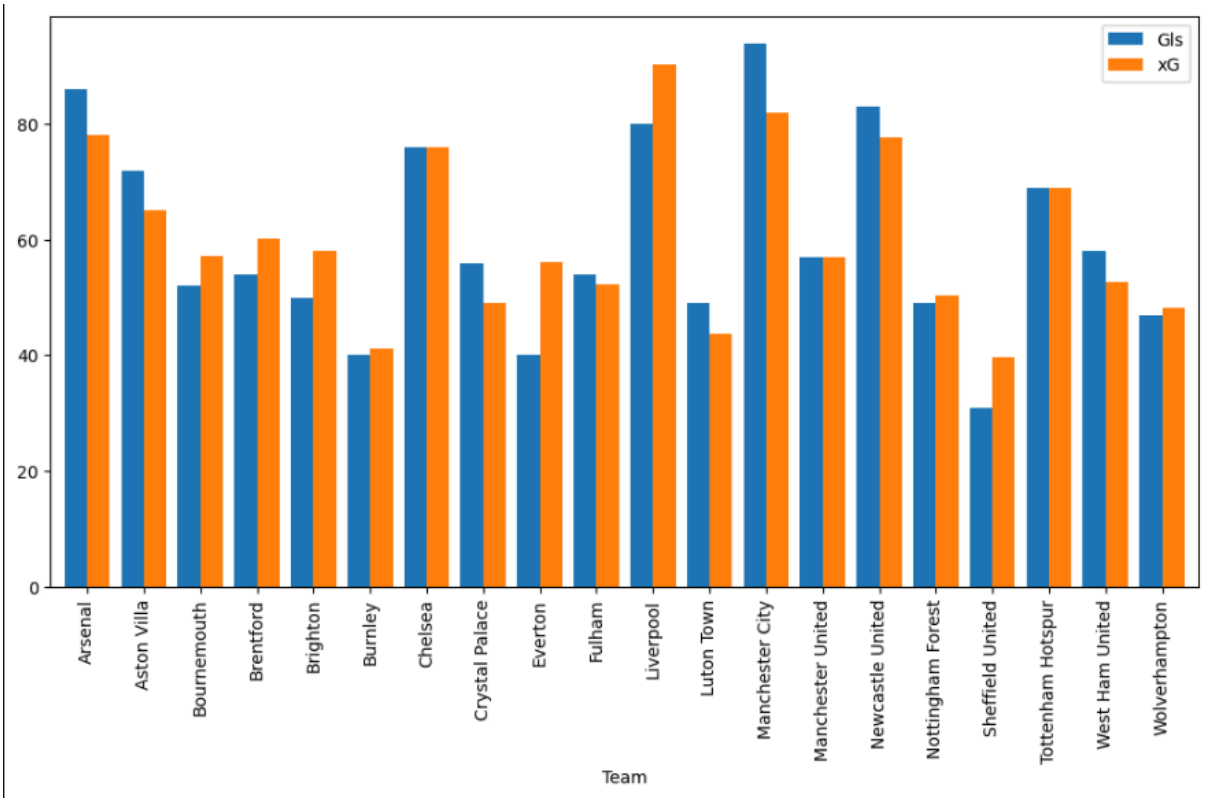
(e.g., direct, possession-based, counterattacking)



The chart shows Manchester City's possession-based style, whereas Arsenal's is more direct, and Fulham's is counter-attacking.

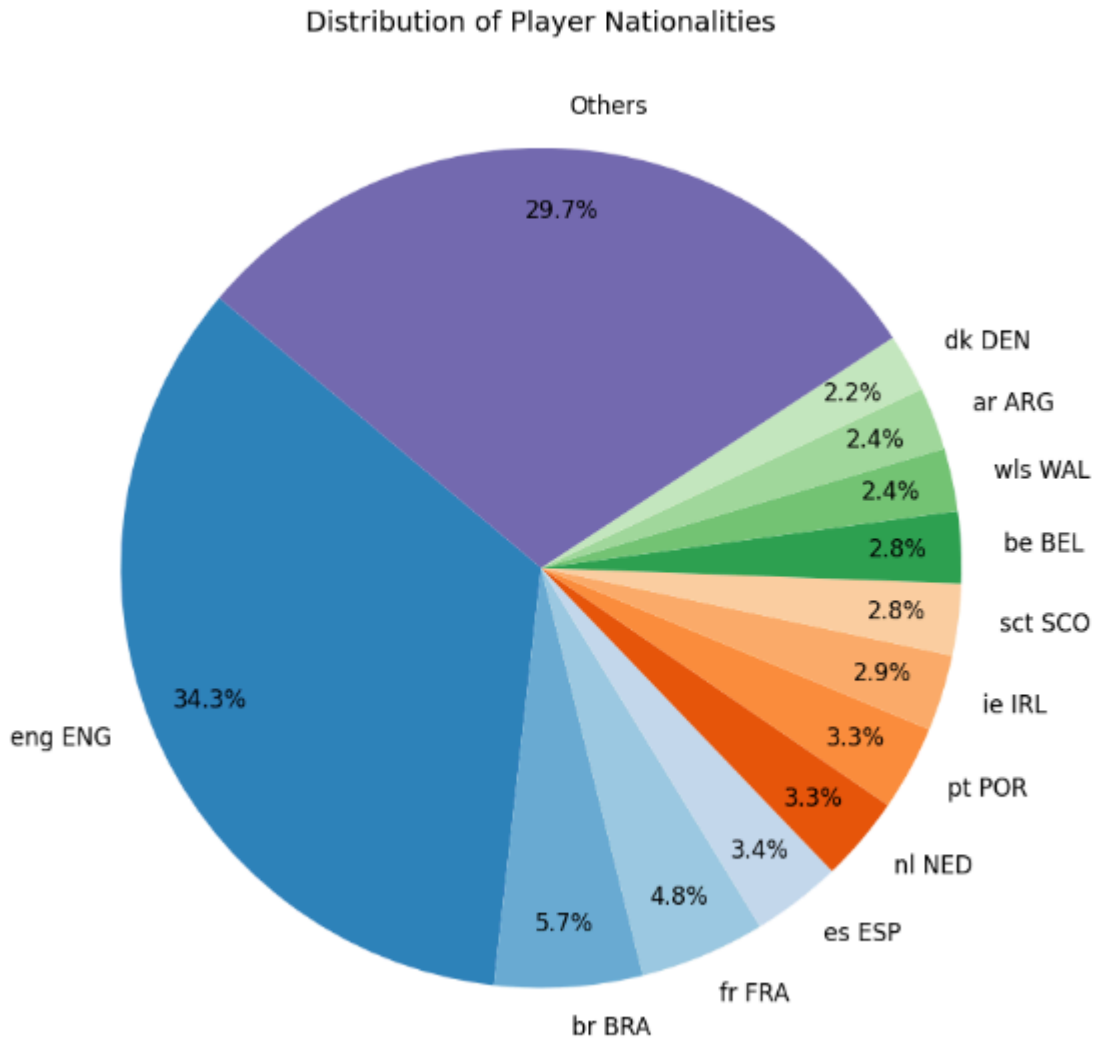
3. Team Efficiency: Actual Goals vs Expected Goals:

We compares Actual Goals (Gls) scored by each team to their Expected Goals (xG), highlighting which teams are overperforming or underperforming relative to their chances.



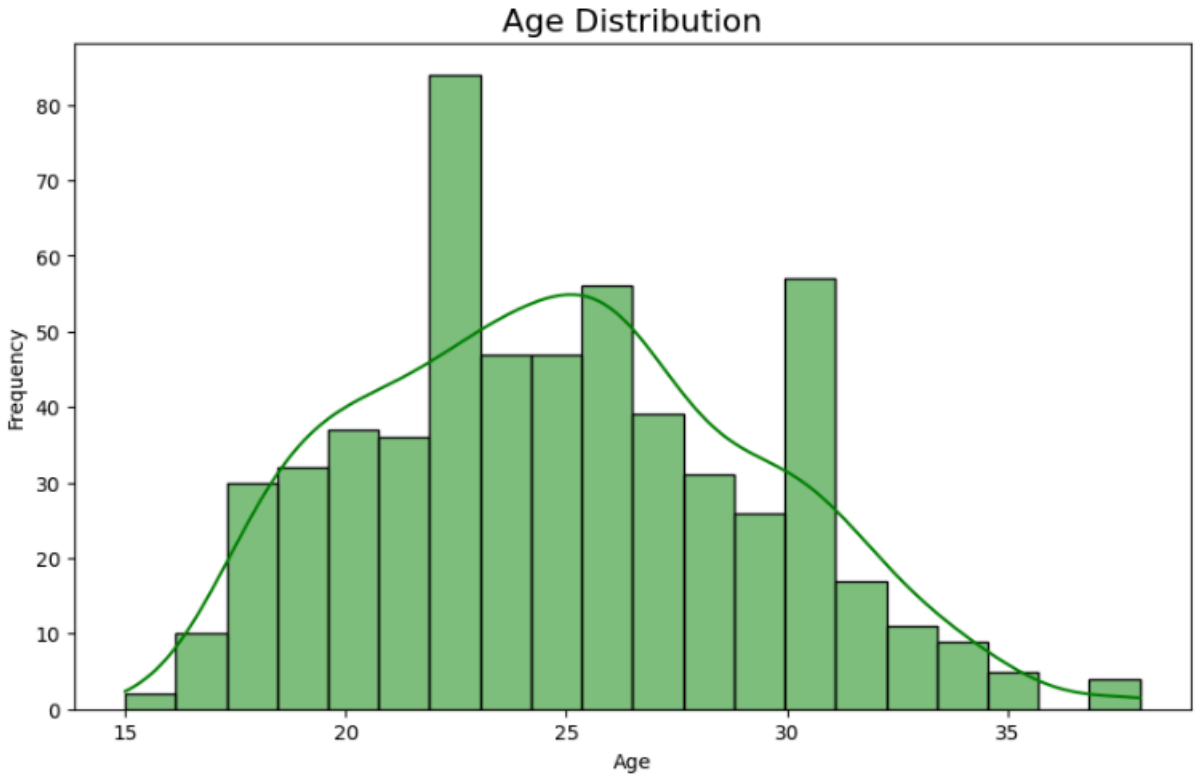
We can see Manchester City is the best team to convert their chances into goals the worst is Liverpool and Everton

4. **Distribution of Player Nationalities:**
   Analyzing the distribution of player nationalities helps uncover the diversity and origin of talent within teams.



5. Age Distribution:

Analyzing the age distribution of players can provide valuable insights that can benefit teams, coaches, analysts, and scouts.



We can observe that the majority of players in the league are between the ages of 23 and 26. Young players may offer energy and vibrancy to the field, but they may lack experience, whereas older players frequently contribute greater expertise, leadership, and strategic thinking.

For more details about the codes it will be in the jupiter notebook

Team Member Name	Contribution Description
مشاري إبراهيم الرحيلي	Data cleaning, handling missing values
عبدالعزيز فهد المطيري	Data analysis, statistical calculations
نواف مساعد الصاعدي	Report writing and conclusion

Conclusion:

Summarise results:

Team Strength and Style of Play:

- Team Goals: Total goals scored by top teams revealed Manchester City's dominance, while others like Arsenal and Newcastle also performed well.
- Progressive Metrics: An analysis of progressive carries, passes, and receives (PrgC, PrgP, PrgR) illustrated team play styles:
    - Manchester City: Possession-heavy
    - Arsenal: Direct attacking approach.
    - Fulham: Counter-attacking with rapid transitions.

Disciplinary Insights:

- **Aggressiveness**:
    - Chelsea had the highest disciplinary infractions, indicating an aggressive playing style.
    - Manchester City displayed superior discipline, reflecting a controlled and strategic approach to the game.

Overperformance and Underperformance:

- A comparison of actual and expected metrics for goals (xG) and assists (xAG) revealed:
    - **Overperformers**: Players who exceeded their expected metrics, showcasing exceptional finishing or playmaking skills.
    - **Underperformers**: Players falling short of expectations, potentially due to inefficiency or poor form.

Discipline vs. Style Correlation:

- A relationship was observed between team discipline (cards) and their play style. For example:
    - Highly disciplined teams like Manchester City often favored a structured, possession-based approach.
    - Aggressive teams like Chelsea had a higher frequency of infractions, possibly indicative of high-tempo or physical strategies.

Challenges encountered:

Although the dataset contains 35 columns covering player attributes and performance metrics, it lacks crucial information such as shots taken, pass accuracy, successful dribbles, tackles won, aerial duels won, and interceptions. These additional details are essential to accurately determine the best player in the league.

Potential real-world applications:

1. Team Performance Optimization:

- **Strategic Decision-Making:** Insights into team styles, strengths, and weaknesses can guide coaches in refining tactics and formations.
- **Training Focus:** Identifying underperforming areas (e.g., low pass accuracy or defensive inefficiencies) helps prioritize training sessions.

2. Player Development and Scouting:

- **Talent Identification:** Advanced metrics like xG, xAG, and progressive actions can uncover hidden talents in lesser-known players.
- **Skill Development:** Pinpointing areas for improvement (e.g., dribbling success or aerial duels) aids in personalized coaching.

3. Match Outcome Predictions:

- **Win, Loss, or Draw**: Predict match outcomes based on historical data, including team performance metrics, player contributions, and head-to-head statistics.
- **Scoreline Forecasting**: Estimate final scores by analyzing expected goals (xG), defensive metrics, and recent form.

4. Player Performance Predictions:

- **Future Performance Trends**:
    - Predict a player's goals, assists, or minutes played based on their historical trends and current form.